Week 6 Structured Notes: Final Review and Array Representation Visualizer

Topic Overview

Week 6 is the final week of the Array Representation curriculum.

In the previous weeks, you learned how arrays work from beginner level up to multidimensional memory formulas. Week 6 connects everything together and prepares you to build the final project: the **Array Representation Visualizer**.

This week mainly covers:

1. Full review of array memory concepts
2. Base address
3. Word size
4. Offset
5. 1D array address formulas
6. 2D row-major formula
7. 2D column-major formula
8. 3D row-major formula
9. 3D column-major formula
10. 4D row-major formula
11. 4D column-major formula
12. Formula comparison
13. Memory diagrams
14. Final project design
15. Menu-driven program structure
16. Step-by-step address calculation
17. Beginner-friendly C++ project code
18. Testing the project
19. Common mistakes
20. Week 6 checkpoint questions

---

1. What You Should Already Know Before Week 6

Before starting Week 6, you should already understand Weeks 1 to 5.

From Week 1

You should know that:

- A variable stores one value.
- An array stores many values under one name.
- Array indexing in C and C++ starts from 0.
- A loop is used to visit all array elements.
- Array elements are stored side by side in memory.
- Accessing one array element is fast.

Example:

```
int A[5] = {2, 4, 6, 8, 10};
```

Logical view:

```
Index:  0   1   2   3   4
Value:  2   4   6   8   10
```

If the base address is 200 and one integer uses 4 bytes:

```
A[0] -> 200
A[1] -> 204
A[2] -> 208
A[3] -> 212
A[4] -> 216
```

From Week 2

You should know that:

- Stack arrays are fixed-size arrays.
- Heap arrays are created dynamically.
- A pointer stores a memory address.
- `malloc` creates heap memory in C.
- `free` releases heap memory in C.
- `new[]` creates a heap array in C++.
- `delete[]` releases a heap array in C++.
- Forgetting to release heap memory causes a memory leak.

Example in C:

```c
int *P = (int*)malloc(5 * sizeof(int));
```

Example in C++:

```cpp
int *P = new int[5];
```

From Week 3

You should know that:

- Arrays need contiguous memory.
- Arrays cannot grow directly.
- Heap arrays can be resized manually.
- Resizing means creating a bigger block, copying old values, freeing the old block, and redirecting the pointer.
- The computer calculates array addresses directly instead of searching one by one.

Main 1D formula:

```
Address of A[i] = L0 + (i * w)
```

Where:

```
L0 = base address
i  = index
w  = word size
```

From Week 4

You should know that:

- A 2D array looks like a table.
- A 2D array has rows and columns.
- `A[i][j]` means row `i`, column `j`.
- A 2D array needs nested loops for full traversal.
- A 2D array looks like a table, but memory stores it as one straight line.
- Row-major stores rows first.
- Column-major stores columns first.

2D row-major formula:

```
Address of A[i][j] = L0 + ((i * N) + j) * w
```

2D column-major formula:

```
Address of A[i][j] = L0 + ((j * M) + i) * w
```

From Week 5

You should know that:

- 3D arrays have three indexes.
- 4D arrays have four indexes.
- Multidimensional arrays still use linear memory.
- Row-major formulas move left to right.
- Column-major formulas move right to left.
- Horner's Rule reduces repeated multiplication.

3D row-major formula:

```
Address = L0 + ((I * M * N) + (J * N) + K) * W
```

3D column-major formula:

```
Address = L0 + ((K * L * M) + (J * L) + I) * W
```

---

2. Main Idea of Week 6

Week 6 is not mainly about learning a new formula.

It is about combining everything you already learned.

The main question of Week 6 is:

```
Given an array and a target element, can you calculate where that element is
stored in memory?
```

That is the purpose of the final project.

The final project proves that you understand arrays as memory layouts, not just as syntax.

---

3. The Universal Address Formula

Almost every formula in this topic follows one main pattern:

```
Address = Base Address + Offset * Word Size
```

Where:

```
Base Address = starting address of the array
Offset       = number of elements skipped from the beginning
Word Size    = number of bytes used by one element
```

Simple meaning:

```
Start from the beginning of the array.
Skip some number of elements.
Convert that skip into bytes.
Add it to the base address.
```

So the real work is finding the **offset**.

Once you know the offset, the address is easy.

---

4. Important Terms

4.1 Base Address

The base address is the starting address of the array.

Usually, it is the address of the first element.

Example:

```
A[0] address = 200
```

So:

```
L0 = 200
```

`L0` means base address.

4.2 Word Size

Word size means the number of bytes used by one array element.

Example:

```
char   -> usually 1 byte
int    -> often 4 bytes
double -> often 8 bytes
```

If an integer takes 4 bytes, then:

```
w = 4
```

4.3 Index

An index is the position number of an array element.

Example:

```
A[3]
```

Here:

```
i = 3
```

In C and C++, indexing starts from 0.

So:

```
A[0] = first element
A[1] = second element
A[2] = third element
A[3] = fourth element
```

4.4 Offset

Offset means the number of elements skipped from the beginning of the array.

Example:

```
A[3]
```

In 0-indexing, to reach `A[3]`, we skip 3 elements:

```
A[0], A[1], A[2]
```

So:

```
offset = 3
```

Then the address is:

```
Address = L0 + offset * w
```

---

5. Formula Review

5.1 1D Array: 0-Indexed Formula

C and C++ use 0-based indexing.

Formula:

```
Address of A[i] = L0 + (i * w)
```

Where:

```
L0 = base address
i  = index
w  = word size
```

Example

Given:

```
L0 = 200
i = 3
w = 4
```

Find the address of `A[3]`.

Formula:

```
Address of A[i] = L0 + (i * w)
```

Substitute values:

```
Address of A[3] = 200 + (3 * 4)
```

Calculate:

```
Address = 200 + 12
Address = 212
```

Answer:

```
Address of A[3] = 212
```

Memory view:

```
A[0] -> 200
A[1] -> 204
A[2] -> 208
A[3] -> 212
A[4] -> 216
```

5.2 1D Array: 1-Indexed Formula

Some textbook examples use 1-based indexing.

In 1-based indexing:

```
A[1] = first element
A[2] = second element
A[3] = third element
```

Formula:

```
Address of A[i] = L0 + ((i - 1) * w)
```

Example

Given:

```
L0 = 500
i = 4
w = 2
```

Find the address of `A[4]`.

Formula:

```
Address of A[i] = L0 + ((i - 1) * w)
```

Substitute values:

```
Address of A[4] = 500 + ((4 - 1) * 2)
```

Calculate:

```
Address = 500 + (3 * 2)
Address = 500 + 6
Address = 506
```

Answer:

```
Address of A[4] = 506
```

5.3 0-Indexed vs 1-Indexed

Given:

```
L0 = 1000
w = 4
i = 3
```

0-indexed calculation

```
Address = 1000 + (3 * 4)
Address = 1012
```

In 0-indexing:

```
A[3] = fourth element
```

1-indexed calculation

```
Address = 1000 + ((3 - 1) * 4)
Address = 1000 + 8
Address = 1008
```

In 1-indexing:

```
A[3] = third element
```

Main difference:

```
The same expression A[3] can mean different positions depending on the indexing
system.
```

For C and C++:

```
Use the 0-indexed formula.
```

---

6. 2D Array Review

A 2D array looks like a table.

Example:

```
int A[3][4];
```

This means:

```
3 rows
4 columns
```

Logical view:

```
1   2   3   4
5   6   7   8
9   10  11  12
```

But memory is still linear:

```
1 2 3 4 5 6 7 8 9 10 11 12
```

Important idea:

```
A 2D array looks like a table, but memory stores it as one straight line.
```

---

7. 2D Row-Major Mapping

Meaning

Row-major mapping stores the array row by row.

Example table:

```
1   2   3   4
5   6   7   8
9   10  11  12
```

Row-major memory order:

```
1 2 3 4 | 5 6 7 8 | 9 10 11 12
```

It stores:

```
Row 0 first
Row 1 second
Row 2 third
```

C and C++ built-in 2D arrays use row-major order.

Formula

```
Address of A[i][j] = L0 + ((i * N) + j) * w
```

Where:

```
L0 = base address
i  = row index
j  = column index
N  = total number of columns
w  = word size
```

Memory aid:

```
Row-major = row first
Offset = i * columns + j
```

Why Row-Major Uses Total Columns

In row-major order, we skip full rows first.

Each full row contains `N` columns.

So:

```
i * N = number of elements skipped from previous rows
j     = number of elements moved inside the current row
```

Example

Given:

```
Array = A[3][4]
L0 = 200
Target = A[1][2]
N = 4
w = 2
```

Formula:

```
Address = L0 + ((i * N) + j) * w
```

Substitute:

```
Address = 200 + ((1 * 4) + 2) * 2
```

Calculate:

```
Address = 200 + (4 + 2) * 2
Address = 200 + 6 * 2
```

```
Address = 200 + 12
Address = 212
```

Answer:

```
Address of A[1][2] = 212
```

---

8. 2D Column-Major Mapping

Meaning

Column-major mapping stores the array column by column.

Example table:

```
1   2   3   4
5   6   7   8
9   10  11  12
```

Column-major memory order:

```
1 5 9 | 2 6 10 | 3 7 11 | 4 8 12
```

It stores:

```
Column 0 first
Column 1 second
Column 2 third
Column 3 fourth
```

Column-major is not the normal built-in 2D array order in C and C++, but it is important for understanding general array representation.

Formula

```
Address of A[i][j] = L0 + ((j * M) + i) * w
```

Where:

```
L0 = base address
i  = row index
j  = column index
M  = total number of rows
w  = word size
```

Memory aid:

```
Column-major = column first
Offset = j * rows + i
```

Why Column-Major Uses Total Rows

In column-major order, we skip full columns first.

Each full column contains `M` rows.

So:

```
j * M = number of elements skipped from previous columns
i     = number of elements moved inside the current column
```

Example

Given:

```
Array = A[3][4]
L0 = 200
Target = A[1][2]
M = 3
w = 2
```

Formula:

```
Address = L0 + ((j * M) + i) * w
```

Substitute:

```
Address = 200 + ((2 * 3) + 1) * 2
```

Calculate:

```
Address = 200 + (6 + 1) * 2
Address = 200 + 7 * 2
Address = 200 + 14
Address = 214
```

Answer:

```
Address of A[1][2] = 214
```

---

9. Why Row-Major and Column-Major Can Give Different Addresses

For the same target:

```
A[1][2]
```

Row-major address:

```
212
```

Column-major address:

```
214
```

Why?

Because the storage order is different.

Row-major memory order:

```
1 2 3 4 5 6 7 8 9 10 11 12
```

Column-major memory order:

```
1 5 9 2 6 10 3 7 11 4 8 12
```

Important sentence:

```
Same logical position can have a different physical address if the storage order
changes.
```

---

10. 3D Array Review

A 3D array has three indexes.

Example:

```
int A[2][3][4];
```

This means:

```
First dimension size  = 2
Second dimension size = 3
Third dimension size  = 4
```

A target element is written as:

```
A[I][J][K]
```

For formulas, use this notation:

```
Array dimensions: A[L][M][N]
Target element:   A[I][J][K]
```

Important:

```
L, M, N = dimension sizes
I, J, K = target indexes
```

Do not mix them.

---

11. 3D Row-Major Formula

Row-major means moving from left to right.

For:

```
A[L][M][N]
Target = A[I][J][K]
```

Formula:

```
Address = L0 + ((I * M * N) + (J * N) + K) * W
```

Where:

```
L0 = base address
I  = first index
J  = second index
K  = third index
M  = second dimension size
N  = third dimension size
W  = word size
```

Understanding the Formula

```
I * M * N
```

This skips full blocks before the target block.

```
J * N
```

This skips full rows inside the current block.

```
K
```

This moves inside the current row.

So:

```
Offset = I * M * N + J * N + K
```

Then:

```
Address = L0 + offset * W
```

Example

Given:

```
Array = A[2][3][4]
Target = A[1][2][3]
L0 = 1000
W = 4
```

Identify dimensions:

```
L = 2
M = 3
N = 4
```

Identify indexes:

```
I = 1
J = 2
K = 3
```

Formula:

```
Address = L0 + ((I * M * N) + (J * N) + K) * W
```

Substitute:

```
Address = 1000 + ((1 * 3 * 4) + (2 * 4) + 3) * 4
```

Calculate:

```
Address = 1000 + (12 + 8 + 3) * 4
Address = 1000 + 23 * 4
Address = 1000 + 92
Address = 1092
```

Answer:

```
Address of A[1][2][3] = 1092
```

---

12. 3D Column-Major Formula

Column-major means moving from right to left.

For:

```
A[L][M][N]
Target = A[I][J][K]
```

Formula:

```
Address = L0 + ((K * L * M) + (J * L) + I) * W
```

Where:

```
L0 = base address
I  = first index
J  = second index
```

```
K  = third index
L  = first dimension size
M  = second dimension size
W  = word size
```

Understanding the Formula

```
K * L * M
```

This skips earlier third-dimension groups.

```
J * L
```

This skips earlier second-dimension groups.

```
I
```

This moves inside the current group.

So:

```
Offset = K * L * M + J * L + I
```

Then:

```
Address = L0 + offset * W
```

Example

Given:

```
Array = A[3][4][5]
Target = A[2][1][3]
L0 = 500
W = 2
```

Identify dimensions:

```
L = 3
M = 4
N = 5
```

Identify indexes:

```
I = 2
J = 1
K = 3
```

Formula:

```
Address = L0 + ((K * L * M) + (J * L) + I) * W
```

Substitute:

```
Address = 500 + ((3 * 3 * 4) + (1 * 3) + 2) * 2
```

Calculate:

```
Address = 500 + (36 + 3 + 2) * 2
Address = 500 + 41 * 2
Address = 500 + 82
Address = 582
```

Answer:

```
Address = 582
```

---

13. 4D Array Review

A 4D array has four indexes.

Example:

```
int A[2][3][4][5];
```

Use this notation:

```
Array dimensions: A[d1][d2][d3][d4]
Target element:   A[i1][i2][i3][i4]
```

Where:

```
d1, d2, d3, d4 = dimension sizes
i1, i2, i3, i4 = target indexes
```

The same rule still applies:

```
Address = L0 + offset * word size
```

---

14. 4D Row-Major Formula

Row-major means moving left to right.

Formula:

```
Address = L0 + (
    i1 * d2 * d3 * d4 +
    i2 * d3 * d4 +
    i3 * d4 +
    i4
) * w
```

Memory aid:

```
Each index is multiplied by all dimensions after it.
```

So:

```
i1 gets d2 * d3 * d4
i2 gets d3 * d4
```

```
i3 gets d4
i4 gets nothing
```

Example

Given:

```
Array = A[2][3][4][5]
Target = A[1][1][2][3]
L0 = 1000
w = 4
```

Formula:

```
Address = L0 + (
    i1*d2*d3*d4 +
    i2*d3*d4 +
    i3*d4 +
    i4
) * w
```

Substitute:

```
Address = 1000 + (
    1*3*4*5 +
    1*4*5 +
    2*5 +
    3
) * 4
```

Calculate:

```
Address = 1000 + (60 + 20 + 10 + 3) * 4
Address = 1000 + 93 * 4
Address = 1000 + 372
Address = 1372
```

Answer:

```
Address = 1372
```

---

15. 4D Column-Major Formula

Column-major means moving right to left.

Formula:

```
Address = L0 + (
    i4 * d1 * d2 * d3 +
    i3 * d1 * d2 +
    i2 * d1 +
    i1
) * w
```

Memory aid:

```
Each index is multiplied by all dimensions before it.
```

So:

```
i4 gets d1 * d2 * d3
i3 gets d1 * d2
i2 gets d1
i1 gets nothing
```

---

16. Row-Major vs Column-Major Formula Building

Row-Major Rule

Row-major moves from left to right.

Rule:

```
Each index is multiplied by the dimensions after it.
```

Example:

```
A[d1][d2][d3][d4]
A[i1][i2][i3][i4]
```

Offset:

```
i1*d2*d3*d4 + i2*d3*d4 + i3*d4 + i4
```

Column-Major Rule

Column-major moves from right to left.

Rule:

```
Each index is multiplied by the dimensions before it.
```

Example:

```
A[d1][d2][d3][d4]
A[i1][i2][i3][i4]
```

Offset:

```
i4*d1*d2*d3 + i3*d1*d2 + i2*d1 + i1
```

---

17. Formula Quick Reference

1D 0-indexed

```
Address of A[i] = L0 + (i * w)
```

1D 1-indexed

```
Address of A[i] = L0 + ((i - 1) * w)
```

2D Row-Major

```
Address of A[i][j] = L0 + ((i * N) + j) * w
```

Where:

```
N = total columns
```

2D Column-Major

```
Address of A[i][j] = L0 + ((j * M) + i) * w
```

Where:

```
M = total rows
```

3D Row-Major

```
Address = L0 + ((I * M * N) + (J * N) + K) * W
```

3D Column-Major

```
Address = L0 + ((K * L * M) + (J * L) + I) * W
```

4D Row-Major

```
Address = L0 + (
    i1*d2*d3*d4 +
    i2*d3*d4 +
    i3*d4 +
```

```
    i4
) * w
```

4D Column-Major

```
Address = L0 + (
    i4*d1*d2*d3 +
    i3*d1*d2 +
    i2*d1 +
    i1
) * w
```

---

18. Final Project: Array Representation Visualizer

18.1 Project Meaning

The final project is a console-based program that calculates array element addresses.

It should ask the user for:

- Array type
- Base address
- Word size
- Dimensions
- Target index or indexes

Then it should print:

- Formula
- Substituted values
- Step-by-step calculation
- Final address

18.2 Why This Project Matters

This project proves that you understand arrays as memory layouts.

A weak program only prints the final answer.

A good program explains the calculation.

Good output should look like this:

```
Array Type: 2D Row-Major
Base Address: 200
Rows: 3
Columns: 4
Target: A[1][2]
Word Size: 2

Formula:
Address = L0 + ((i * N) + j) * w

Calculation:
Address = 200 + ((1 * 4) + 2) * 2
Address = 200 + 6 * 2
Address = 200 + 12
Address = 212
```

---

19. Project Menu Design

The program should be menu-driven.

A simple beginner menu:

```
===== Array Representation Visualizer =====
1. 1D 0-indexed address
2. 1D 1-indexed address
3. 2D Row-Major address
4. 2D Column-Major address
5. 3D Row-Major address
6. 3D Column-Major address
7. Exit
Enter your choice:
```

Optional advanced menu:

```
8. 4D Row-Major address
9. 4D Column-Major address
```

Beginner advice:

```
First build 1D, 2D, and 3D.
Add 4D only after the smaller formulas work.
```

---

20. How to Build the Project Safely

Do not build the entire project at once.

Build it in small steps.

Recommended order:

```
Step 1: Create the menu
Step 2: Add 1D 0-indexed formula
Step 3: Add 1D 1-indexed formula
Step 4: Add 2D row-major formula
Step 5: Add 2D column-major formula
Step 6: Add 3D row-major formula
Step 7: Add 3D column-major formula
Step 8: Test each formula manually
Step 9: Add input validation
Step 10: Add 4D formulas if needed
```

Why this order is safe:

```
If you write everything at once, debugging becomes hard.
If you add one formula at a time, mistakes are easier to find.
```

---

21. Beginner-Friendly C++ Project Code

This version supports:

- 1D 0-indexed address
- 1D 1-indexed address
- 2D row-major address
- 2D column-major address
- 3D row-major address
- 3D column-major address

```cpp
#include <iostream>
using namespace std;

void oneDZeroIndexed() {
    int L0, i, w;

    cout << "Enter base address L0: ";
    cin >> L0;

    cout << "Enter index i: ";
    cin >> i;

    cout << "Enter word size w: ";
    cin >> w;

    int offset = i;
    int address = L0 + offset * w;

    cout << "\nFormula:\n";
    cout << "Address of A[i] = L0 + (i * w)\n";

    cout << "\nSubstitution:\n";
    cout << "Address of A[" << i << "] = " << L0 << " + (" << i << " * " << w
<< ")\n";

    cout << "\nCalculation:\n";
    cout << "Offset = " << offset << "\n";
    cout << "Address = " << L0 << " + " << offset << " * " << w << "\n";
    cout << "Address = " << address << "\n";
}

void oneDOneIndexed() {
    int L0, i, w;

    cout << "Enter base address L0: ";
    cin >> L0;

    cout << "Enter index i: ";
    cin >> i;

    cout << "Enter word size w: ";
    cin >> w;

    int offset = i - 1;
    int address = L0 + offset * w;

    cout << "\nFormula:\n";
```

```
    cout << "Address of A[i] = L0 + ((i - 1) * w)\n";

    cout << "\nSubstitution:\n";
    cout << "Address of A[" << i << "] = " << L0 << " + ((" << i << " - 1) * "
<< w << ")\n";

    cout << "\nCalculation:\n";
    cout << "Offset = " << i << " - 1 = " << offset << "\n";
    cout << "Address = " << L0 << " + " << offset << " * " << w << "\n";
    cout << "Address = " << address << "\n";
}

void twoDRowMajor() {
    int L0, i, j, N, w;

    cout << "Enter base address L0: ";
    cin >> L0;

    cout << "Enter row index i: ";
    cin >> i;

    cout << "Enter column index j: ";
    cin >> j;

    cout << "Enter total columns N: ";
    cin >> N;

    cout << "Enter word size w: ";
    cin >> w;

    int offset = (i * N) + j;
    int address = L0 + offset * w;

    cout << "\nFormula:\n";
    cout << "Address of A[i][j] = L0 + ((i * N) + j) * w\n";

    cout << "\nSubstitution:\n";
    cout << "Address = " << L0 << " + ((" << i << " * " << N << ") + " << j <<
") * " << w << "\n";

    cout << "\nCalculation:\n";
    cout << "Offset = (" << i << " * " << N << ") + " << j << " = " << offset
<< "\n";
    cout << "Address = " << L0 << " + " << offset << " * " << w << "\n";
    cout << "Address = " << address << "\n";
}

void twoDColumnMajor() {
```

```
    int L0, i, j, M, w;

    cout << "Enter base address L0: ";
    cin >> L0;

    cout << "Enter row index i: ";
    cin >> i;

    cout << "Enter column index j: ";
    cin >> j;

    cout << "Enter total rows M: ";
    cin >> M;

    cout << "Enter word size w: ";
    cin >> w;

    int offset = (j * M) + i;
    int address = L0 + offset * w;

    cout << "\nFormula:\n";
    cout << "Address of A[i][j] = L0 + ((j * M) + i) * w\n";

    cout << "\nSubstitution:\n";
    cout << "Address = " << L0 << " + ((" << j << " * " << M << ") + " << i <<
") * " << w << "\n";

    cout << "\nCalculation:\n";
    cout << "Offset = (" << j << " * " << M << ") + " << i << " = " << offset
<< "\n";
    cout << "Address = " << L0 << " + " << offset << " * " << w << "\n";
    cout << "Address = " << address << "\n";
}

void threeDRowMajor() {
    int L0, I, J, K, M, N, W;

    cout << "Enter base address L0: ";
    cin >> L0;

    cout << "Enter index I: ";
    cin >> I;

    cout << "Enter index J: ";
    cin >> J;

    cout << "Enter index K: ";
    cin >> K;
```

```
    cout << "Enter second dimension size M: ";
    cin >> M;

    cout << "Enter third dimension size N: ";
    cin >> N;

    cout << "Enter word size W: ";
    cin >> W;

    int offset = (I * M * N) + (J * N) + K;
    int address = L0 + offset * W;

    cout << "\nFormula:\n";
    cout << "Address = L0 + ((I * M * N) + (J * N) + K) * W\n";

    cout << "\nSubstitution:\n";
    cout << "Address = " << L0 << " + ((" << I << " * " << M << " * " << N <<
") + (" << J << " * " << N << ") + " << K << ") * " << W << "\n";

    cout << "\nCalculation:\n";
    cout << "Offset = " << offset << "\n";
    cout << "Address = " << L0 << " + " << offset << " * " << W << "\n";
    cout << "Address = " << address << "\n";
}

void threeDColumnMajor() {
    int L0, I, J, K, L, M, W;

    cout << "Enter base address L0: ";
    cin >> L0;

    cout << "Enter index I: ";
    cin >> I;

    cout << "Enter index J: ";
    cin >> J;

    cout << "Enter index K: ";
    cin >> K;

    cout << "Enter first dimension size L: ";
    cin >> L;

    cout << "Enter second dimension size M: ";
    cin >> M;

    cout << "Enter word size W: ";
```

```
    cin >> W;

    int offset = (K * L * M) + (J * L) + I;
    int address = L0 + offset * W;

    cout << "\nFormula:\n";
    cout << "Address = L0 + ((K * L * M) + (J * L) + I) * W\n";

    cout << "\nSubstitution:\n";
    cout << "Address = " << L0 << " + ((" << K << " * " << L << " * " << M <<
") + (" << J << " * " << L << ") + " << I << ") * " << W << "\n";

    cout << "\nCalculation:\n";
    cout << "Offset = " << offset << "\n";
    cout << "Address = " << L0 << " + " << offset << " * " << W << "\n";
    cout << "Address = " << address << "\n";
}

int main() {
    int choice;

    do {
        cout << "\n===== Array Representation Visualizer =====\n";
        cout << "1. 1D 0-indexed address\n";
        cout << "2. 1D 1-indexed address\n";
        cout << "3. 2D Row-Major address\n";
        cout << "4. 2D Column-Major address\n";
        cout << "5. 3D Row-Major address\n";
        cout << "6. 3D Column-Major address\n";
        cout << "7. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                oneDZeroIndexed();
                break;

            case 2:
                oneDOneIndexed();
                break;

            case 3:
                twoDRowMajor();
                break;

            case 4:
                twoDColumnMajor();
```

```
                break;

            case 5:
                threeDRowMajor();
                break;

            case 6:
                threeDColumnMajor();
                break;

            case 7:
                cout << "Exiting program.\n";
                break;

            default:
                cout << "Invalid choice. Try again.\n";
        }

    } while (choice != 7);

    return 0;
}
```

---

22. How to Test the Project

Testing means checking whether the program gives the correct answer.

Do not trust the program immediately.

First, calculate manually.

Then compare with the program output.

Test 1: 1D 0-indexed

Given:

```
L0 = 200
i = 3
w = 4
```

Expected calculation:

```
Address = 200 + (3 * 4)
Address = 212
```

Expected answer:

```
212
```

Test 2: 2D Row-Major

Given:

```
L0 = 200
A[3][4]
Target = A[1][2]
N = 4
w = 2
```

Expected calculation:

```
Address = 200 + ((1 * 4) + 2) * 2
Address = 200 + 6 * 2
Address = 212
```

Expected answer:

```
212
```

Test 3: 2D Column-Major

Given:

```
L0 = 200
A[3][4]
Target = A[1][2]
```

```
M = 3
w = 2
```

Expected calculation:

```
Address = 200 + ((2 * 3) + 1) * 2
Address = 200 + 7 * 2
Address = 214
```

Expected answer:

```
214
```

Test 4: 3D Row-Major

Given:

```
A[2][3][4]
Target = A[1][2][3]
L0 = 1000
W = 4
```

Expected calculation:

```
Address = 1000 + ((1 * 3 * 4) + (2 * 4) + 3) * 4
Address = 1000 + (12 + 8 + 3) * 4
Address = 1000 + 23 * 4
Address = 1092
```

Expected answer:

```
1092
```

---

23. Input Validation

Input validation means checking whether the user entered valid values.

Examples of invalid input:

```
Negative index
Negative word size
Column index outside the array
Row index outside the array
Invalid menu choice
```

Example:

```
A[3][4]
```

Valid row indexes:

```
0, 1, 2
```

Valid column indexes:

```
0, 1, 2, 3
```

Wrong:

```
A[3][0]
```

Why wrong?

```
Row index 3 is outside the array.
The last valid row index is 2.
```

For a beginner version, add input validation after the formulas work.

---

24. Common Week 6 Mistakes

Mistake 1: Memorizing formulas without understanding

Bad approach:

```
Trying to memorize all formulas separately.
```

Better approach:

```
Understand how to build the offset.
```

Mistake 2: Forgetting the word size

Wrong:

```
Address = L0 + offset
```

Correct:

```
Address = L0 + offset * word_size
```

The offset counts elements.

The address needs bytes.

That is why we multiply by word size.

Mistake 3: Mixing up rows and columns

For:

```
A[3][4]
```

Correct:

```
3 rows
4 columns
```

Wrong:

```
4 rows
3 columns
```

Mistake 4: Using rows in the row-major formula

Wrong:

```
Address = L0 + ((i * M) + j) * w
```

Correct:

```
Address = L0 + ((i * N) + j) * w
```

In row-major:

```
N = total columns
```

Mistake 5: Using columns in the column-major formula

Wrong:

```
Address = L0 + ((j * N) + i) * w
```

Correct:

```
Address = L0 + ((j * M) + i) * w
```

In column-major:

```
M = total rows
```

Mistake 6: Confusing dimension sizes and indexes

For:

```
A[L][M][N]
A[I][J][K]
```

Correct:

```
L, M, N = sizes
I, J, K = positions
```

Wrong:

```
Thinking L and I are the same thing.
```

Mistake 7: Printing only the final answer

Weak output:

```
Address = 212
```

Better output:

```
Formula:
Address = L0 + ((i * N) + j) * w

Substitution:
Address = 200 + ((1 * 4) + 2) * 2

Calculation:
Address = 200 + 6 * 2
Address = 212
```

The project should explain the calculation, not just output the answer.

---

25. Week 6 Summary

By the end of Week 6, you should understand that:

- Week 6 is a review and project week.
- The final project is the Array Representation Visualizer.
- The main formula pattern is `Address = L0 + offset * w`.
- The base address is the starting address of the array.
- Word size is the number of bytes used by one element.
- Offset means how many elements are skipped.
- 1D 0-indexed arrays use `L0 + (i * w)`.
- 1D 1-indexed arrays use `L0 + ((i - 1) * w)`.
- 2D row-major uses `L0 + ((i * N) + j) * w`.
- 2D column-major uses `L0 + ((j * M) + i) * w`.
- 3D row-major uses `L0 + ((I * M * N) + (J * N) + K) * W`.
- 3D column-major uses `L0 + ((K * L * M) + (J * L) + I) * W`.
- 4D formulas follow the same direction rules.
- Row-major moves left to right.
- Column-major moves right to left.
- Multidimensional arrays are still stored in linear memory.
- The final project should print the formula, substituted values, calculation steps, and final address.

---

26. Week 6 Checkpoint Questions

Try answering these without looking at the notes.

1. What is the main purpose of Week 6?
2. What is the Array Representation Visualizer?
3. What should the final project ask the user to enter?
4. What should the final project print as output?
5. What is a base address?
6. What is word size?
7. What is an offset?
8. What is the universal address formula?
9. What is the 1D 0-indexed formula?
10. What is the 1D 1-indexed formula?
11. Why does C/C++ use the 0-indexed formula?
12. What is the 2D row-major formula?
13. What does `N` mean in the row-major formula?
14. What is the 2D column-major formula?
15. What does `M` mean in the column-major formula?
16. Why can row-major and column-major give different addresses?
17. What is the 3D row-major formula?
18. What is the 3D column-major formula?

19. What is the 4D row-major formula?
20. What is the 4D column-major formula?
21. In row-major order, do we move left to right or right to left?
22. In column-major order, do we move left to right or right to left?
23. Why is memory still linear for 2D, 3D, and 4D arrays?
24. Why should the project show calculation steps?
25. Why should you build the project one formula at a time?

---

27. Practice Tasks

Task 1: 1D 0-Indexed Address

Given:

```
L0 = 300
i = 4
w = 4
```

Find the address of `A[4]`.

Formula:

```
Address = L0 + (i * w)
```

Solution:

```
Address = 300 + (4 * 4)
Address = 300 + 16
Address = 316
```

Answer:

```
316
```

Task 2: 1D 1-Indexed Address

Given:

```
L0 = 500
i = 3
w = 2
```

Formula:

```
Address = L0 + ((i - 1) * w)
```

Solution:

```
Address = 500 + ((3 - 1) * 2)
Address = 500 + 4
Address = 504
```

Answer:

```
504
```

Task 3: 2D Row-Major Address

Given:

```
A[4][5]
Target = A[2][3]
L0 = 500
w = 4
```

Formula:

```
Address = L0 + ((i * N) + j) * w
```

Here:

```
i = 2
j = 3
N = 5
```

Solution:

```
Address = 500 + ((2 * 5) + 3) * 4
Address = 500 + 13 * 4
Address = 552
```

Answer:

```
552
```

Task 4: 2D Column-Major Address

Given:

```
A[4][5]
Target = A[2][3]
L0 = 500
w = 4
```

Formula:

```
Address = L0 + ((j * M) + i) * w
```

Here:

```
i = 2
j = 3
M = 4
```

Solution:

```
Address = 500 + ((3 * 4) + 2) * 4
Address = 500 + 14 * 4
Address = 556
```

Answer:

```
556
```

Task 5: 3D Row-Major Address

Given:

```
A[2][3][4]
Target = A[1][1][2]
L0 = 200
W = 4
```

Formula:

```
Address = L0 + ((I * M * N) + (J * N) + K) * W
```

Here:

```
I = 1
J = 1
K = 2
M = 3
N = 4
```

Solution:

```
Address = 200 + ((1 * 3 * 4) + (1 * 4) + 2) * 4
Address = 200 + (12 + 4 + 2) * 4
Address = 200 + 18 * 4
Address = 272
```

Answer:

```
272
```

Task 6: 3D Column-Major Address

Given:

```
A[2][3][4]
Target = A[1][1][2]
L0 = 200
W = 4
```

Formula:

```
Address = L0 + ((K * L * M) + (J * L) + I) * W
```

Here:

```
I = 1
J = 1
K = 2
L = 2
M = 3
```

Solution:

```
Address = 200 + ((2 * 2 * 3) + (1 * 2) + 1) * 4
Address = 200 + (12 + 2 + 1) * 4
Address = 200 + 15 * 4
Address = 260
```

Answer:

```
260
```

---

28. Simple Final Memory Aid

```
Address = L0 + offset * word_size
```

For 1D:

```
offset = i
```

For 2D row-major:

```
offset = i * columns + j
```

For 2D column-major:

```
offset = j * rows + i
```

For 3D row-major:

```
offset = I * M * N + J * N + K
```

For 3D column-major:

```
offset = K * L * M + J * L + I
```

For 4D row-major:

```
offset = i1*d2*d3*d4 + i2*d3*d4 + i3*d4 + i4
```

For 4D column-major:

```
offset = i4*d1*d2*d3 + i3*d1*d2 + i2*d1 + i1
```

Direction rule:

```
Row-major    = left to right
Column-major = right to left
```

Most important Week 6 idea:

```
Do not only memorize formulas.
Understand how many elements are skipped.
Then calculate the address from that offset.
```